Shader (alpha_outline_URP) 使用说明

1. 包含文件

请确保以下 4 个文件放置在项目中的同一目录下，切勿遗漏 `.meta` 文件，否则会导致材质丢失 Shader 关联：

- `alpha_outline.shader` & `.meta` (着色器核心代码，**基于URP 2D 渲染管线编写**)
- `alpha_outline.mat` & `.meta` (已配置好颜色参数的材质球成品)

2. 核心功能概述

目前交互状态（描边 + 半透明颜色叠加）：

- **合法与非法指示器**：绿色的半透明 (用于合法位置高亮) / 红色的半透明 (用于非法位置高亮)（鼠标点选）
- **选中状态**：高亮边框 (鼠标悬停)

目前已在材质球面板暴露了颜色的自定义接口，程序端无需在代码中硬编码颜色值，也无需动态替换整个 Material。

3. 图片导入设置

由于描边需要在图片原有透明区域外扩进行渲染，请确保在使用此材质（如建筑、单元格等）的原图 `Import Settings` 中：

- **Mesh Type 须设置为 `Full Rect`**

> **注意：** 使用默认的 `Tight` 会导致边缘描边被网格切断，从而出现显示不完整或无反应的情况。（虽然现在效果也不是很好吧...可能是计算边缘的算法不太对...）

4. 程序说明（理是这个理，但不确定C#对不对，是gemini说的）

通过代码获取该材质，并根据逻辑变化修改内部的 `_State` 浮点值，即可切换显示状态。在 Unity Editor 的 Inspector 面板中，将准备好的 `alpha_outline.mat` 材质球拖给对应物体的 `SpriteRenderer` 组件的 Material。

示例代码：

```
// 1. 获取材质引用 (这会实例化当前挂载的 alpha_outline 材质)
Material mat = GetComponent<SpriteRenderer>().material;

// 2. 切换状态 (传入枚举值对应的浮点数: 0=Normal, 1=Hover, 2=Valid, 3=Invalid)
mat.SetFloat("_State", 1);
```